

Tool: Logging Standard

Apa: standar structured logging — format field konsisten biar tiap incident bisa ditelusur. **Kapan dipake:** setup service (baseline) + tiap endpoint/auto-action (SOP-SA-05). **Framework:** 12-Factor App (faktor XI: log as event stream ke stdout), ISO/IEC 27001 (logging & monitoring), OWASP (gak leak secret/PII). **Output:** standar di `tools/` ; log produksi ke observability stack (@levi).

Aturan

- **JSON terstruktur** ke **stdout** (12-Factor) — bukan teks bebas, bukan file manual.
- **Logger proven** (Pino/winston) + **log level** (debug/info/warn/error). Debug gak nyebrang ke prod.
- **NO `console.log`** di prod.
- **Redact secret/PII** (password, token, nomor kartu) — never log mentah.
- Tiap log error → ada **error code** dari `error-code-table.md`.

Field wajib per request log

Field	Isi	Kenapa
<code>timestamp</code>	ISO 8601	urutan kronologi
<code>level</code>	info/warn/error	filter severity
<code>request_id</code>	UUID per request	korelasi 1 request lintas-log
<code>user_id</code>	id user (atau <code>anon</code>)	siapa yang ngelakuin
<code>action</code>	<code>payment.created</code> , <code>auth.login</code>	apa yang terjadi
<code>outcome</code>	<code>success</code> / <code>error:<CODE></code>	hasil
<code>latency_ms</code>	durasi	RED metric (Duration)

Auto-action (Data SACRED — SOP-SA-04)

- Tiap auto-action (auto-settle, dedup, retry) → log eksplisit: **siapa-trigger, kapan, kenapa**.
- Dampak user rasain → **notif user-facing** juga (gak cukup log doang). **Silent = haram**.

TEMPLATE (shape log entry)

```
{  
  "timestamp": "2026-05-29T10:00:00.123Z",
```

```
"level": "info",
"request_id": "7f3e-...",
"user_id": "user-42",
"action": "<domain.verb>",
"outcome": "success",
"latency_ms": 38,
"meta": { "...": "field domain, NO secret/PII mentah" }
}
```

CONTOH TERISI (dari proyek contoh — flow payment + auto-settle)

```
// 1. User submit payment
{
  "timestamp": "2026-05-29T10:00:00.123Z",
  "level": "info",
  "request_id": "7f3e-aa01",
  "user_id": "user-42",
  "action": "payment.created",
  "outcome": "success",
  "latency_ms": 41,
  "meta": { "order_id": "abc-123", "amount": 150000, "method": "qris",
            "idempotency_key": "7f3e-aa01", "status": "PENDING" }
}

// 2. Idempotent replay (double-tap) — gak double-charge
{
  "timestamp": "2026-05-29T10:00:01.050Z",
  "level": "warn",
  "request_id": "7f3e-aa02",
  "user_id": "user-42",
  "action": "payment.created",
  "outcome": "error:PAYMENT_ALREADY_PROCESSED",
  "latency_ms": 9,
  "meta": { "idempotency_key": "7f3e-aa01", "returned_existing": "payment-9001" }
}

// 3. AUTO-SETTLE pas webhook payment masuk — EKSPLISIT (Data SACRED, gak silent)
{
  "timestamp": "2026-05-29T10:03:22.000Z",
  "level": "info",
  "request_id": "wh-5521",
  "user_id": "system:payment-webhook",
  "action": "payment.settled",
  "outcome": "success",
  "latency_ms": 55,
  "meta": { "payment_id": "payment-9001", "from_status": "PENDING", "to_status": "SETTLED",
            "trigger": "qris_webhook", "event_appended": true,
            "owner_notified": true }
}
```

3 hal yang ditunjukkan contoh ini: 1. **Trace-ability** — `request_id` bikin 1 flow payment bisa dirangkai (kontrol SA-C5). 2. **Idempotency kelihatan di log** — `replay = warn` + `returned_existing`, bukan transaksi baru (kontrol SA-C6). 3. **Auto-settle EKSPLISIT** — ada entry `payment.settled` dengan `from→to status` + `owner_notified: true`. Status uang gak pernah di-overwrite diam-diam; tiap transisi 1 event (Data SACRED + Tata "auto-everything silent = haram").

Yang **gak boleh**: `console.log("settled")` tanpa field, atau auto-settle yang cuma ubah kolom status tanpa event + tanpa notif. Itu silent = haram.